LLM Quiz Prep — Master Study Guide

Core Innovations · Training · Post-Training (RLHF/DPO/GRPO) · Mixture of Experts · Parallelism

Compiled from course decks

Contents

How to Use This Guide

You already know Transformers (self-attention, Q/K/V, encoder/decoder, multi-head attention). This guide picks up from there and covers everything else in your decks, in the order you'll likely be tested:

1. **Core Innovations in Recent LLMs** — normalization, activations, positional encoding/RoPE, KV cache, attention variants (MQA/GQA/MLA), FlashAttention, PagedAttention, decoding strategies
2. **Training I** — Pretraining and Supervised Fine-Tuning (SFT)
3. **Post-Training II** — Preference tuning, preference data, RL formulation, RLHF, Reward Modeling (Bradley-Terry)
4. **Post-Training III** — PPO (advantage, clipping, KL penalty), Best-of-N, DPO, GRPO, RLVR
5. **Mixture of Experts** — gating, routing, load balancing, capacity, Switch Transformer
6. **Parallelism** — data/pipeline/tensor/3D parallelism

Each part ends with **key takeaways**. The back of the guide has an **MCQ bank**, **short-answer theory questions**, and a **tricky-questions/gotchas** section — read those twice.

Part 1 — Core Innovations in Recent LLMs (Architecture)

1.1 Why Normalization?

Problem: Internal Covariate Shift. As a network trains, the distribution of inputs to each hidden layer keeps changing (because earlier layers' weights keep updating). This forces later layers to constantly re-adapt, slowing training.

Fix idea: normalize each layer's outputs to have **mean = 0** and **variance = 1**.

- Mean = 0 → centers activations around the origin (not consistently biased + or −)
- Variance = 1 → standardizes the spread so features are on comparable scales

Batch Normalization (BatchNorm)

- Normalizes across the **batch dimension** — for each feature, compute mean/std across all examples in the mini-batch.
- Formula (per feature): $\hat{x} = \dfrac{x - \mu_{batch}}{\sqrt{\sigma^2_{batch} + \epsilon}}$, then scale/shift with learnable γ, β.
- Differentiable → can backprop through it.
- **Weakness:** depends on batch statistics. Poor fit for sequence models (variable-length sequences, small batches for long sequences) — batch composition changes what "mean/std" means batch to batch.

Layer Normalization (LayerNorm)

- Normalizes across the **feature dimension**, per single example — independent of batch size/composition. This is why it's preferred for sequence models / Transformers.
- Has a **re-centering and re-scaling invariance** property: output doesn't change if you shift/scale all input features by the same amount.

RMSNorm (1st Enhancement over LayerNorm)

- **Motivation:** LayerNorm's mean-subtraction (re-centering) step is often unnecessary and even harmful:
  - Q and K come from linear projections; their dot products naturally produce positive and negative values — mean subtraction can remove useful absolute information, couples feature representations, and adds redundant computation.
  - At scale (billions of params), every normalization layer's FLOPs/latency add up — removing the mean-computation step saves real compute.
- **What RMSNorm does:** drops re-centering (no mean subtraction), keeps only re-scaling — normalizes by the **root mean square** of the activations.
- $\text{RMSNorm}(x) = \dfrac{x}{\sqrt{\frac{1}{n}\sum x_i^2 + \epsilon}} \cdot \gamma$
- **Simpler, parameter-efficient, faster, and used in most modern LLMs (LLaMA, etc.)** — enhances training stability and generalization across model sizes.

Quick contrast table:

	Normalizes over	Removes mean?	Best for
BatchNorm	batch dimension	Yes	CNNs / fixed-size batches
LayerNorm	feature dimension (per example)	Yes	Sequences (original Transformer)

	Normalizes over	Removes mean?	Best for
RMSNorm	feature dimension (per example)	**No** — scale only	Modern LLMs (efficiency)

1.2 Activation Functions: Toward SwiGLU

Problems with basic activations:

- **ReLU(x) = max(0, x):** discards all negative information (hard gate, zero gradient for x<0 → "dying ReLU").
- **Sigmoid:** saturates at extremes → vanishing gradients; outputs are always positive, not zero-centered.

Gated Linear Unit (GLU)

- A **learnable gate** that dynamically controls how much of each feature passes through — instead of a hard on/off (ReLU), it's a soft, learned gate.
- $\text{GLU}(x) = (xW_1) \otimes \sigma(xW_2)$ — one branch carries information, the other (passed through sigmoid) decides how much of it flows through.
- Intuition given in lecture: x_1 = water flowing through a pipe (information), x_2 = valve handle (gate), $\sigma(x_2)$ = how open the valve is. Or: x_1 = sunlight, x_2 = lens tint, $\sigma(x_2)$ = amount of tint.
- **Problem with GLU:** it still uses **Sigmoid** for the gate, which re-introduces saturation — some features get almost fully blocked.

Swish / SiLU

- $\text{Swish}(x) = x \cdot \sigma(x)$ — a smooth version of ReLU/GLU-gate, with a **non-zero gradient for negative values** (unlike ReLU which fully kills negative inputs).
- More expressive → richer feature transformations.

SwiGLU (2nd Enhancement)

- Replace the sigmoid gate in GLU with **Swish**: $\text{SwiGLU}(x) = \text{Swish}(xW_1) \otimes (xW_2)$ (also written $\text{Swish}(xW) \otimes (xV)$).
- Used in the FFN of LLaMA and most modern LLMs, replacing plain ReLU FFNs.
- **Why it wins:** avoids ReLU's hard zeroing, avoids GLU's sigmoid saturation, is smooth/differentiable everywhere, and empirically improves LLM quality.

1.3 Positional Encoding

The problem: Self-attention computes relationships based purely on **content** (token embeddings) — it is **permutation-invariant** (order doesn't affect the output structurally). The model has no built-in notion of "cat" coming before or after "mouse" — it's a "bag of words" unless we inject position info. We also want **no recurrence** (unlike RNN/LSTM which are inherently sequential).

Sinusoidal Positional Encoding (original Transformer)

- Add a positional vector to each token embedding.
- $PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$, $PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$
- Each dimension corresponds to a sinusoid; wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$.

- Each row of the PE matrix = the encoding vector for one position, with 512 (or d_{model}) values between -1 and 1, added elementwise to the token embedding.
- **Limitation:** these are **absolute** position embeddings — the model essentially memorizes "this is position 23" instead of learning relative relationships ("this word is 3 steps after that one"). Two sentences of different lengths give the same words different *combined* embeddings even if their *relative* structure is similar. This is a problem because language is fundamentally relational (dependencies span relative positions).
- Also: position info is **added to** token embeddings rather than integrated into the attention computation itself ("positional disentanglement") — harder for the model to learn nuanced sequence-structure patterns.

RoPE — Rotary Positional Embeddings (3rd Enhancement)

- **Core idea:** instead of *adding* a position vector, **rotate** the Query and Key vectors by a position-dependent **angle**. Rotation preserves vector length (magnitude) but changes orientation.
- The dot product between a rotated Q_m (position m) and rotated K_n (position n) becomes a function of **only the angle difference** $\theta_m - \theta_n$ — i.e., depends on **relative distance** $(m-n)$, not absolute position.
- This means: the attention score between two tokens depends on how far apart they are, not where they sit in the sequence.
- Words close together (small $|m-n|$) → smaller angle difference → higher dot product → stronger attention score (naturally encodes locality/relatedness).
- **Used in:** LLaMA, GPT-NeoX, most modern open LLMs. Solves the "no relative information" and "positional disentanglement" problems of sinusoidal PE.

Example distinguishing absolute vs relative: "The cat sat on the mat" → distance(cat, mat) = 4 tokens. "On the mat, the cat sat quietly" → distance(cat, mat) = 2 tokens. Absolute PE would encode raw positions and "memorize" them; RoPE naturally captures that these are different relative distances via rotation angle.

1.4 Next-Token Prediction & the Motivation for KV Cache

- Training: given input `[SOS] Love that can quickly seize the gentle heart`, target is the same sequence shifted by one, ending in `[EOS]` — the model learns to predict the next token at every position in parallel (teacher forcing).
- Inference: **autoregressive** — at T=1 you only have `[SOS]`, predict "Love"; at T=2 input is `[SOS] Love`, predict "that"; etc. Each step needs the full previous context to decide the next token.
- **Key inefficiency:** at every inference step, you're only interested in the *new* token, but self-attention recomputes over the *entire* prefix each time — recomputing Keys/Values for tokens you've already processed is wasteful.
- **KV Cache:** cache the Key and Value vectors of previously seen tokens; at each new step, only compute Q/K/V for the *new* token, reuse cached K/V for the rest. Massively reduces redundant computation during autoregressive generation.

1.5 GPU Memory Bottleneck (Why efficient attention matters)

- Modern GPUs (e.g., A100) have huge compute throughput (~19.5 TFLOPs) but are **memory-bandwidth limited** (~2 TB/s) — the real bottleneck is moving data (tensors) in/out of memory, not raw arithmetic.
- Implication: reuse tensors across many operations rather than repeatedly reading/writing them; minimize tensor movement, maximize data reuse.

1.6 Multi-Query Attention (MQA) and Grouped-Query Attention (GQA)

- **Standard Multi-Head Attention (MHA):** every head has its own Q, K, V projections — most memory-bandwidth-hungry (large KV cache).
- **Multi-Query Attention (MQA):** all query heads **share a single** K and V head — drastically cuts KV-cache memory, but can hurt quality (too aggressive sharing).
- **Grouped-Query Attention (GQA):** a middle ground — heads are divided into **groups**, and each group shares its own K/V (instead of one shared K/V for everything, or one K/V per head).
  - **Benefits:** lowers parameter count, reduces memory bandwidth (smaller KV cache) vs. full MHA, while retaining more quality than MQA.
  - GQA has become the **new standard** replacement for MHA in most modern LLMs (LLaMA 2/3, etc.).

1.7 Paged Attention (vLLM's memory manager)

- **Problem:** naive KV-cache allocation reserves a large contiguous memory block per sequence (sized for max possible length), wasting memory (internal fragmentation) since most sequences are shorter.
- **Solution (inspired by OS virtual memory / paging):** split the KV cache into fixed-size **blocks** (logical KV blocks, e.g., block size = 4 tokens). Maintain a **page table** mapping logical blocks → physical memory blocks, allocated on demand.
- Enables near-zero waste, and lets multiple requests **share** memory pools (e.g., shared prompt prefixes).
- This is the core memory-management technique behind **vLLM**.

1.8 Sliding Window Attention

- Restrict each token to attend only to itself and the previous W tokens (a fixed **window**), instead of the full history.
- Example: window size 3 → each token attends to itself + previous 2 tokens only.
- **Pros:** reduces number of dot products → improves compute efficiency (training & inference).
- **Cons:** may degrade quality — long-range dependencies aren't explicitly captured in a single layer.
- **Mitigation:** stacking multiple Transformer layers lets information propagate beyond the local window over depth — similar to how receptive fields grow in CNNs. This aligns with the intuition that nearby text is usually most relevant (Chapter 5 words relate mostly to nearby Chapter 5 text, less to Chapter 1).

1.9 FlashAttention

- **Goal:** reduce memory *accesses* (not FLOPs) during attention computation — attention is bottlenecked by reads/writes to slow **HBM** (High Bandwidth Memory), not by the compute itself.
- GPU memory hierarchy: small/fast **SRAM** vs. large/slow **HBM**. FlashAttention reorganizes computation to touch HBM as little as possible.
- **Two key optimizations:**
  1. **Tiling:** break Q, K, V into blocks small enough to fit in SRAM, so each value is read from HBM only **once** (linear accesses) instead of repeatedly (quadratic accesses) — classic tiling for matrix multiplication.
  2. **Recomputation:** avoid storing the full (huge) attention matrix; recompute needed intermediate values on the fly during the backward pass instead of storing them all — trades a bit of extra compute for large memory savings.
- Softmax needs care when computed block-by-block: track running statistics **m** (max) and **l** (normalizer) per block, and rescale/combine as new blocks are processed (numerically-stable

"online softmax").
- **Net effect:** FlashAttention is *exact* (not an approximation) — same output as standard attention, just computed with far fewer slow-memory accesses → much faster, especially at long sequence lengths.

1.10 vLLM Inference Engine

- Open-source, high-throughput, low-latency LLM/VLM inference engine.
- Drop-in replacement for the OpenAI API.
- Supports LLaMA 2/3, Qwen, Mistral, Mixtral, Gemma, DeepSeek, Phi, etc.
- Written in Python/CUDA with custom kernels; its efficiency largely comes from **PagedAttention**.

1.11 Decoding Strategies

- After the final layer + softmax, you get a probability distribution over the **entire vocabulary** at each generation step (e.g., "mat": 0.25, "floor": 0.12, ...). Deciding **which token to actually emit** — the decoding strategy — has a large effect on output quality, diversity, and coherence.

Greedy Decoding

- $y_t = \arg\max P(y \mid y_1, \ldots, y_{t-1}, X)$ — always pick the single highest-probability token at each step.
- **Strengths:** simple, fast, deterministic; good for tasks with a clear "right answer" (translation, summarization).
- **Weakness:** can be repetitive/suboptimal globally (locally-greedy choices aren't always globally best) and lacks diversity.

(Other strategies like sampling, top-k, top-p/nucleus, beam search follow the same "pick from the distribution" framing even if not explicitly detailed in the slides — know that greedy is deterministic/argmax-based and is the baseline others improve on for diversity/quality trade-offs.)

Speculative Decoding

- **Motivation:** standard generation emits **one token at a time**, which is slow; also, some prediction steps are "easy" and some are "hard."
- **Idea:** run **two models in parallel** —
  - **Target model:** the large model you actually want (e.g., Llama-70B).
  - **Draft model:** a smaller, lightweight model (e.g., Llama-7B) that quickly proposes several candidate next tokens.
- The target model then verifies the draft's proposed tokens in a single forward pass:
  - **Accepted tokens (green):** draft's guess matches what the target model would have picked — kept, no extra work needed for those positions.
  - **Rejected tokens (red):** draft's guess is discarded.
  - **Corrected tokens (blue):** target model supplies its own correct token at the point of rejection.
- **Benefit:** since verifying multiple draft tokens in one target-model pass is cheaper than generating them one-by-one with the target model, this **speeds up inference** without changing the output distribution (same quality as running the target model alone).

1.12 Multi-Head Latent Attention (MLA)

- Another KV-cache memory-saving strategy (used in DeepSeek models), complementary to GQA.

- Instead of *sharing* K/V heads across groups (like GQA), MLA **compresses** the Key and Value tensors into a **lower-dimensional latent vector** before caching.
- At inference time, the compressed latent vectors are **projected back up** to their original size when needed for attention computation.
- **Trade-off:** a small amount of extra computation (projecting back up) for a much larger reduction in **memory** usage — since memory (not compute) is the main bottleneck in long-context autoregressive inference.
- Equivalent reformulation: instead of "up-project then attend in the large Q,K space" (costly), you can **down-project queries and attend directly in the compressed latent space** (cheaper) — mathematically equivalent, computationally cheaper.
- **Key takeaway line from the deck:** *"For the same compute, it's better to have more, smaller attention than less, larger attention."*

Part 1 Key Takeaways

- RMSNorm > LayerNorm > BatchNorm for modern LLMs (drops re-centering, keeps re-scaling, cheaper).
- SwiGLU = Swish-gated GLU; fixes ReLU's hard-zeroing and GLU's sigmoid-saturation.
- RoPE encodes **relative** position via rotation of Q/K vectors — solves absolute-PE's limitations.
- KV Cache avoids recomputing K/V for already-seen tokens during autoregressive decoding.
- MQA (1 shared KV) → GQA (grouped shared KV, the modern standard) → MLA (compressed latent KV) are all about shrinking the KV cache / memory bandwidth.
- PagedAttention (vLLM) = OS-style paging for KV cache memory management.
- Sliding Window Attention trades some long-range info for compute efficiency; depth recovers some long-range reach.
- FlashAttention is exact, not approximate — it saves memory *accesses* via tiling + recomputation, not FLOPs.
- Speculative decoding uses a cheap draft model + expensive target model to speed up generation without changing output quality.

Part 2 — Training I: Pretraining & Supervised Fine-Tuning (SFT)

2.1 The Three Stages of LLM Training

1. **Pre-training**
2. **Instruction tuning** (a form of SFT)
3. **Reinforcement Learning / Alignment with Human Feedback**

2.2 Pretraining

- **Purpose:** build a "base model" with broad language understanding/generation — not specialized to one task.
- **Data scale:** hundreds of billions to trillions of tokens (Common Crawl, Wikipedia, etc.).
- **Self-supervision:** no human labels needed — model predicts missing/future tokens from raw text itself.
- **Result:** a model with "basic knowledge" of language, code, facts, etc., but **no notion of how to be a helpful assistant.**

Pretrained model behavior example: Ask "Can I put my teddy bear in the washer?" → a *raw* pretrained model tends to continue the text rather than answer helpfully (e.g., starts describing what teddy bears are made of, in a Wikipedia-continuation style) — it completes text, it doesn't "answer."

2.3 The Remedy: Fine-tuning

Initialized model → **Pretraining** → base model with "basic knowledge" → **Fine-tuning** → model tuned for specific tasks/behaviors.

2.4 Supervised Fine-Tuning (SFT)

- **SFT = Supervised FineTuning.** Idea: change model *behavior* by tuning its weights using labeled input/output pairs.
- **Strategy:** collect (input, desired-output) pairs ("SFT data"); train with the standard next-token-prediction objective, but now conditioned on high-quality demonstrations.
- **Special case — Instruction Tuning:** SFT specifically on instruction-following data. Goal: "graduate" the raw LM into a helpful assistant that follows instructions, not just continues text.
- **Objective function:** still next-token prediction — but now over curated instruction→response demonstrations rather than raw web text.
- **Data mixtures:** assistant dialogs, synthetic instructions, math/reasoning/code data, safety-alignment data, etc. — mix of human-written and synthetic data.

Behavior after instruction tuning: "Can I put my teddy bear in the washer?" → model now answers directly: *"No, it might get damaged. Try hand washing instead."*

2.5 Scale of SFT Data — Surprisingly Small

- InstructGPT used only ~13,000 instruction-response pairs — tiny compared to pretraining's billions of tokens.
- **Why this works:** the model doesn't need to learn *new knowledge* during SFT — that's already baked into the pretrained weights. SFT only needs to teach a **new mode of interaction**: "read instruction → produce a direct response," a relatively simple behavioral shift compared to knowledge acquisition.

- Example dataset sizes: GPT-3 SFT ≈ 13 thousand examples; LLaMA 3 SFT ≈ 10 million examples. (Modern open datasets: Alpaca ~52K, Dolly ~15K, OpenAssistant ~160K; typical range ~10K–100K.)

2.6 What Matters in SFT Data

- **Quality over quantity:** each example must clearly demonstrate the desired behavior — mediocre demonstrations → mediocre assistant.
- **Diversity:** must cover the full range of target behaviors — factual QA, creative writing, code, refusals of harmful requests, multi-turn conversation, expressing uncertainty, following formats.
- **Consistency:** annotators must agree on what "good" looks like — high inter-annotator agreement → more coherent model.

2.7 Duration / Training Dynamics

- SFT typically runs **2–5 epochs** over the instruction dataset.
- **Over-training risk:** the model **memorizes** specific response patterns rather than generalizing (e.g., always starting refusals with the exact verbatim phrase "I'm not able to help with that" instead of learning the *general principle* of when/how to refuse).
- Early stopping based on validation performance is important.

2.8 What SFT Achieves (Behavioral shift example)

- Before SFT: asked "Explain photosynthesis simply," a raw model produces a Wikipedia-style continuation that doesn't actually simplify or stop appropriately.
- After SFT: the model directly answers, uses simple language (because "simply" was in the instruction), gives a structured explanation, and stops when done.

2.9 Challenges of SFT

- Needs very high-quality data.
- Sensitive to the **prompt distribution** used in training (may not generalize to unseen prompt types).
- Generalization is hard to guarantee.
- Difficult to evaluate.
- Computationally expensive.

2.10 Benchmarks

Benchmark	What it measures
MMLU (Massive Multitask Language Understanding)	broad multi-domain academic knowledge (science, history, law, etc.)
ARC-Challenge (AI2 Reasoning Challenge)	grade-school science reasoning, hard multiple-choice, requires inference
GSM8K (Grade School Math 8K)	multi-step mathematical word-problem solving
HumanEval	writing correct Python functions from natural-language specs

- **Validity note:** it's recommended to check whether a model was trained on the test set/task when comparing across models (data contamination concerns).

2.11 Evaluation Challenges (esp. leaderboard/voting-based evaluation)

- **Cold-start problem:** new models get few user ratings early on, so they look worse purely from lack of exposure/feedback, not lower quality.
- **Easy to "rig":** models can learn to produce overly polite/flashy answers to farm higher ratings without actually being more correct.
- **User inability to assess factuality:** a confident-but-wrong answer (e.g., medical) may still get high ratings.
- **Personal preference bias:** raters differ (short vs. detailed answers) → skews global rankings; non-representative rater distribution.
- **Safety penalization:** a safe refusal ("I can't help with that") often gets rated lower than a helpful-but-unsafe answer — evaluation doesn't naturally reward safety.
- **Benefit despite all this:** it puts a number on "vibes" — useful even if imperfect. *Evaluation is a genuinely hard problem in itself.*

Part 2 Key Takeaways

- Pretraining = broad knowledge via self-supervised next-token prediction on massive raw text.
- SFT = behavioral alignment via a *small*, high-quality, diverse, consistent labeled dataset — teaches interaction *style*, not new knowledge.
- Over-training on SFT → memorization of exact phrasing rather than general principles.
- MMLU / ARC / GSM8K / HumanEval each target a different capability (knowledge / reasoning / math / code).
- Evaluation (esp. leaderboards) has systematic biases: cold-start, rigging, factuality blindness, preference bias, safety penalization.

Part 3 — Post-Training II: Preference Tuning & Reward Modeling

3.1 "Alignment" with Human Intents

An AI is considered **aligned** if it is **helpful, honest, and harmless** (the classic 3H framework).

3.2 Why Preference Tuning?

- **Context:** SFT teaches a model to *follow instructions*, but "misbehavior" that shows up isn't necessarily the model being "wrong" per se — SFT is sensitive to its training distribution, and poor/biased SFT data can make the model behave undesirably (e.g., "I'd suggest you don't spend much time with your teddy bear at all" as a bad response).
- **Idea:** collect **preference pairs** (a better response vs. a worse response for the same prompt) and train the model on *which one is better*, rather than only on positive demonstrations.
- **Preference tuning teaches the model *which response is better***, aligning behavior with human expectations — reshaping behavior of an already pretrained+SFT model, **not teaching new reasoning/knowledge**.

Why comparisons are the better signal:

- Comparing two responses ("A is better than B") is **easier and more reliable** for humans than generating the single best response from scratch.
- SFT is sensitive to training-distribution quality/bias.
- High-quality preference data is still expensive/hard to scale, but is generally cheaper/more consistent to collect than perfect demonstrations.
- (Silver lining: a model "misbehaving" after SFT can be a useful signal to go back and check SFT data quality.)

3.3 Preference Data — Three Formats

Given an observation = (prompt, response):

Format	Description
Pointwise	each response gets an absolute score (e.g., Obs1=0.4, Obs2=0.9, Obs3=0.1, Obs4=0.2)
Pairwise	relative comparisons between pairs (Obs1 < Obs2, Obs1 > Obs3, Obs2 > Obs3, …)
Listwise	a full ranking over multiple responses (1st, 2nd, 3rd, 4th, …)

- Pairwise is the most common in practice (most reward-model / RLHF pipelines use pairwise comparisons).

3.4 Recipe to Get (Pairwise) Preference Data

1. **Generate a pair of responses** for the *same* prompt.
   - Input via real chat logs / a reference distribution.
   - Output via the SFT model — synthetic generations / rewrites.
2. **Label** the pair.
   - **Human rating**, or
   - **Proxies:** LLM-as-a-judge, BLEU, ROUGE, etc.
   - Variants: binary scale (strictly "better/worse") vs. a "nuanced" (graded) scale.

3.5 RL Formulation (General, then for LLMs)

Standard RL loop: **Agent** takes an **Action** in an **Environment**, observes a **State**, receives a **Reward**, guided by a **Policy**.

Mapping onto LLMs:

- **Policy** = the LLM itself (π_θ).
- **State** = the input/prompt so far (tokens generated up to now).
- **Action** = the next token to generate.
- **Reward** = a score reflecting human preference (from a reward model).
- **Environment** = trivial for LLMs — mostly just appending the chosen token to the sequence.

Idea: learn π_θ so that it aligns with human preferences — i.e., higher-probability outputs should be outputs humans prefer.

3.6 RLHF = Reinforcement Learning from Human Feedback

(Ouyang et al., 2022 — *"Training language models to follow instructions with human feedback"*, the InstructGPT paper.)

Why RLHF — what fine-tuning (SFT) alone cannot reach

- **SFT teaches imitation** — "produce outputs like these."
- **RLHF teaches preference** — "prefer good outputs over worse ones."
- These are **mathematically different objectives** — SFT optimizes likelihood of exact demonstrations; RLHF optimizes for higher *relative* reward.

The intuition: Demonstrations vs. Comparisons

- Fine-tuning approach = show 10,000 well-written emails, say "write like these."
- RLHF approach = show pairs of emails, say "this one is better — figure out why."
- **Key asymmetry:** *recognition is cheaper than generation.* A human can quickly say "A is better than B," but writing the ideal response from scratch is slower, harder, less consistent. RLHF lets us harvest this cheaper signal at scale. **This single asymmetry is the entire economic/statistical case for RLHF.**

Why RLHF specifically (what pure next-token prediction misses)

A pretrained LLM is an expert at predicting the next token of internet text — **not** the same as being a useful assistant. Next-token prediction alone does **not** train the model to:

- **Follow instructions** the way a user actually expects (not just as disguised text-completion).
- **Decline harmful requests** — refuse unsafe prompts even if the pretraining corpus contains compliant examples of similar requests.
- **Balance trade-offs** — helpfulness, honesty, harmlessness often conflict; RLHF lets us teach the trade-off.
- **Calibrate length** — be concise when brevity is valued, thorough when depth is needed.

RLHF Overview — Two Steps

1. **Step 1 — Reward Modeling:** learn to distinguish good responses from bad ones.
   - Input: (prompt, response). Output: a scalar/quantitative score.
2. **Step 2 — Reinforcement Learning:** use the reward model to align the policy (the LLM).
   - Input: prompt. Output: a response, optimized to maximize expected reward under the RM.

3.7 Step 1: Reward Modeling — the Bradley-Terry Model

Idea: we don't know the *true* underlying reward function r^* that captures real human preferences — but we assume it exists, and model the probability a human prefers y_1 over y_2 given prompt x via the **Bradley-Terry (BT)** model (Bradley & Terry, 1952 — originally for ranking via pairwise comparisons).

$$p^*(y_1 \succ y_2 \mid x) = \sigma(r^*(x, y_1) - r^*(x, y_2))$$

where σ is the logistic (sigmoid) function.

Three cases — let $\Delta = r^*(x, y_1) - r^*(x, y_2)$:

- **Δ = 0** (equal rewards) → $\sigma(0) = 0.5$ → 50/50 toss-up.
- **Δ > 0** (y_1 better) → $\sigma(\Delta) > 0.5$; larger gap → probability approaches 1. (e.g., Δ=3 → ≈0.95)
- **Δ < 0** (y_1 worse) → $\sigma(\Delta) < 0.5$; y_1 can still occasionally "win" — this is exactly how the model captures human noise/disagreement. (e.g., Δ=−3 → ≈0.05)

Training the Reward Model

We train a **parameterized** reward model r_φ to approximate the unknown r^*: it's trained as a **binary classifier** on preference pairs, by plugging r_φ into the Bradley-Terry formula and minimizing the **negative log-likelihood** over the preference dataset $\mathcal{D} = \{(x, y_w, y_l)\}$ (winner/loser pairs):

$$\mathcal{L}_{RM}(\varphi) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\Big[\log \sigma(r_\varphi(x, y_w) - r_\varphi(x, y_l))\Big]$$

- This is the **same logistic loss** as Bradley-Terry MLE.
- **What it does:** minimizing it pushes the RM to assign ≥0.5 probability to the human-chosen ("winner") response; the score difference acts like a confidence that the winner should win, trained to match observed human judgments.

Reward Model — Data & Architecture

- **Data:** O(10,000) preference observations; label = human rating (the "HF" in RLHF).
- **Model:** a pretrained LLM with a **classification head** instead of a next-token-prediction head (i.e., outputs a single scalar reward, not a distribution over vocabulary). Often encoder-only style (e.g., BERT-like, via a `[CLS]` projection) or a decoder model repurposed with a scalar output head.

Part 3 Key Takeaways

- Preference tuning reshapes behavior via *comparisons*, not new demonstrations — cheaper/more reliable signal than "write the ideal answer."
- Preference data comes in pointwise / pairwise / listwise forms; pairwise is most common.
- RLHF = (1) train a reward model via Bradley-Terry-style pairwise loss, then (2) use RL to optimize the policy against that reward model.
- The core asymmetry justifying RLHF: **recognition (comparing) is cheaper than generation (writing the ideal answer).**
- Bradley-Terry: $p^*(y_1 \succ y_2|x) = \sigma(r^*(x, y_1) - r^*(x, y_2))$ — memorize this formula and what Δ=0, Δ>0, Δ<0 mean.

Part 4 — Post-Training III: RL Algorithms (PPO, DPO, GRPO, RLVR)

4.1 Step 2: Reinforcement Learning — Setting Up the Objective

Once we have a reward model r_φ, use it to update the LLM's weights via RL: **penalize bad answers, promote good answers.**

- **Given:**
  - A **reference/base policy** $\pi_{ref}(y|x)$ — typically the instruction-tuned model, the starting point of alignment.
  - A **reward model** $r(x, y)$.
- **Aim:** find a policy $\pi^*_\theta(y|x)$ that:
  1. Generates outputs with **high reward**, **and**
  2. **Stays close** to the reference policy π_{ref}.

Why "high reward" alone isn't enough — why we need closeness to π_{ref}

Reward models are **imperfect** — trained on a limited distribution of natural outputs. If you optimize *purely* for reward:

- **Reward hacking:** the policy can find outputs that get high RM score but are actually meaningless/low quality (e.g., long, confident, hallucinated answers that the RM mistakenly scores highly).
- **Mode collapse / diversity loss:** an input can have many valid outputs (e.g., "write a poem" has infinitely many good poems); pure reward maximization can collapse probability onto a single "best" output style instead of preserving diversity.
- **Staying close to** π_{ref} preserves diversity and prevents these failure modes.

The RLHF/PPO Objective (conceptual)

$$\max_\theta \ \mathbb{E}_{y\sim\pi_\theta}\big[r(x,y)\big] \ - \ \beta \cdot D_{KL}\big(\pi_\theta(\cdot|x) \,\|\, \pi_{ref}(\cdot|x)\big)$$

- First term: **maximize reward**.
- Second term: **penalty for deviating too far from the reference/base model** (a KL-divergence constraint/regularizer).
- β (sometimes λ): hyperparameter controlling how strongly the KL penalty is enforced.
  - **Higher** $\beta \rightarrow$ policy must stay closer to π_{ref} (more conservative).
  - Acts as regularization, preventing the LLM from "forgetting" its original knowledge or degenerating into low-quality outputs.

4.2 PPO — Proximal Policy Optimization

(Schulman et al., 2017)

- **Data:** O(100,000) observations; label = score given by the reward model.
- **Model:** initialized at the SFT model.
- **Training:** update policy weights via the objective above — maximize reward while penalizing deviation from the base model via KL divergence.

PPO Actually Optimizes *Advantage*, Not Raw Reward

- **Advantage ≈ Reward − Baseline**, where the **Baseline (Value Function)** is the model's own internal prediction of what reward it *expected* to get for that state — trained jointly with the policy, using the actual reward as its target label.

- **Why:** using advantage instead of raw reward reduces variance in the gradient estimate and tells the model whether an action was better or worse **than expected**, not just whether the absolute reward was high or low.
- The **value function is token-level**: it estimates expected future reward at *every* token position, not just at the end of the sequence.

Variation 1 — PPO-Clip

- **Idea:** clip the ratio between the new and old policy to prevent large, destabilizing updates from one iteration to the next.
- **Ratio:** $r_t(\theta) = \dfrac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ — measures how different the current policy is from the policy used to generate the data.
- **Clipped objective:**

$$L^{CLIP}(\theta) = \mathbb{E}_t\Big[\min\left(r_t(\theta)A_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\,A_t\right)\Big]$$

- **Intuition:**
  - If an action performed better than expected (positive advantage), we want to increase its probability — but if r_t becomes too large, the *unclipped* term would over-boost it; the `min()` with the clipped version **prevents over-boosting good actions**.
  - If an action performed worse than expected (negative advantage) and r_t becomes too small, the `min()` similarly **prevents over-penalizing bad actions**.
  - Net effect: "improve the policy, but don't move too far from the old one — increase/decrease probability, but only a little at each update." Prevents the "big jump → collapse/instability" failure mode.

Variation 2 — PPO-KL Penalty

- **Idea:** instead of (or alongside) clipping, directly **penalize the difference** in policy distributions via a KL-divergence term added to the loss.
- **Terminology:**
  - `old` = the policy from the *previous RL iteration* (used for the importance-sampling ratio).
  - `ref` = the *original base model* (used for the "don't drift from the base model" constraint).
  - **Nowadays, the KL penalty is typically computed with respect to `ref` (the base model)**, not `old`.

Limitations of PPO

- Needs **up to 4 models simultaneously**: the policy (being trained), the value/baseline model, the reward model, and the reference/base model — heavy memory/compute cost. Raises the question: "is it worth it?"

4.3 Best-of-N (BoN) — A Workaround That Skips RL

- **Idea:** skip the RL optimization step entirely; just leverage the reward model at **inference/generation time**.
- **Strategy:**
  1. Given a prompt, generate **several** candidate outputs using the SFT model.
  2. **Score** each candidate with the reward model.
  3. **Return the highest-scoring** candidate.

- **Limitation (motivates DPO):** Best-of-N is **costly at inference time** (must generate + score N candidates for every single query) — this motivates asking: *"why don't we train in a supervised fashion instead?"*

4.4 DPO — Direct Preference Optimization

(Rafailov et al., 2023 — *"Direct Preference Optimization: Your Language Model is Secretly a Reward Model"*)

- **Core idea:** algebraically **rewrite** the RLHF objective/loss so it can be optimized in a **purely supervised** way, directly on preference pairs — **no separate reward model needed**, and **no RL loop needed** (no sampling, no PPO machinery).
- Mathematically, DPO shows that the optimal policy under the standard RLHF KL-constrained reward-maximization objective has a closed form relating π_θ and π_{ref} to an *implicit* reward — so you can substitute this implicit reward directly into the Bradley-Terry preference loss and optimize π_θ end-to-end via a supervised classification-style loss (essentially the same DPO loss shape as the Bradley-Terry loss, but with reward replaced by the log-ratio of policy to reference-policy probabilities).
- **Key properties:**
  - **No $r(x, y)$ term** — no need to train a separate reward model.
  - Operates **directly** on preference data (prompt, chosen response, rejected response).
  - **Similar to the Bradley-Terry formulation, but with a special/implicit kind of reward** derived from the policy itself (hence "your language model is secretly a reward model").

PPO-based RLHF vs. DPO

	RLHF (PPO)	DPO
Ease of implementation	Multi-stage training	Supervised learning (simpler)
Extra models needed	Reward model + value model + base/reference model	Only the base/reference model
Performance	No absolute consensus — task-dependent, sensitive to implementation	Same

(Xu et al., 2024 — *"Is DPO Superior to PPO for LLM Alignment? A Comprehensive Study"* — no clean universal winner; it varies by task and implementation details.)

Behavior progression example (teddy bear question)

- Pretrained only: gives an unrelated continuation.
- – Instruction tuned: answers directly but perhaps abruptly ("No, it might get damaged. Try hand washing instead.")
- – Instruction tuned + preference tuned: more helpful/nuanced tone ("It's better not to. Your teddy could get hurt! A gentle hand wash is safer.")

4.5 GRPO — Group Relative Policy Optimization

(Shao et al., 2024 — DeepSeekMath paper — *"DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models"*)

- **Common RL algorithm used for reasoning models** (e.g., DeepSeek-R1-style training).

- **Core idea:** GRPO **replaces the separate value function/baseline** (used in PPO) with a **relative ranking among multiple sampled responses** for the *same* prompt — making training **simpler and more memory-efficient** (no separate value/critic model needed).
- **Advantage ≈ Reward − Average(reward of the group)** — i.e., for a group of G sampled responses to the same prompt, each response's advantage is how much better/worse its reward is compared to the group's mean reward (often also normalized by the group's std).
- Same overall objective shape as PPO otherwise: "don't deviate too much from the old/base model" + "maximize advantage" (with clipping as in PPO-Clip typically retained).
- **Why this matters for reasoning tasks:** removes the need to train and maintain a separate value model (which is hard to get right for long chain-of-thought reasoning traces), and directly leverages the fact that **for a given prompt, you can cheaply sample many candidate solutions and compare them to each other.**

GRPO vs PPO — Quick Contrast

	PPO	GRPO
Baseline / value estimate	Separate learned value function (critic model)	Average reward of a **group** of sampled responses to the same prompt
Models needed	Policy + Value + Reward + Reference (4)	Policy + Reward + Reference (no separate value model)
Memory efficiency	Lower	Higher
Typical use case	General RLHF alignment	Reasoning tasks (math, code)

4.6 Reinforcement Learning with Verifiable Rewards (RLVR)

- Primarily used for/evaluated on problems that are **verifiable in nature** — e.g., math and coding.
- **"Verifiable" means:** we have access to either
  1. a **ground-truth answer** for the problem, or
  2. a **rules-based technique** that can automatically verify correctness (e.g., unit tests for code, checking a final numeric answer for math).
- This removes the need for a learned, imperfect reward model in these domains — reward is computed directly/objectively (e.g., "did the final answer match the ground truth?" / "did the code pass the tests?"), which is more reliable than a learned RM and is a big reason GRPO+RLVR became popular for training reasoning models.

Part 4 Key Takeaways

- PPO objective = maximize reward − β·KL(policy‖reference); actually optimizes **advantage** (reward − baseline/value function), not raw reward.
- PPO-Clip clips the policy ratio to prevent destabilizing large updates; PPO-KL directly penalizes divergence from the reference model.
- PPO needs 4 models (policy, value, reward, reference) → expensive.
- Best-of-N skips training entirely, but is costly at **inference** time (generate+score N samples per query).
- **DPO** rewrites the RLHF loss as a supervised loss directly on preference pairs — no reward model, no RL loop, only needs the reference model.
- **GRPO** replaces PPO's value function with the **average reward of a sampled group** — simpler, more memory-efficient, popular for reasoning-model training.
- **RLVR** uses ground-truth/rule-based verifiable rewards (math/code) instead of a learned reward model.

Part 5 — Mixture of Experts (MoE)

5.1 Motivation — Why MoE?

- **Neural scaling laws:** performance improves smoothly as compute, dataset size, or model size increases (Kaplan et al., 2020). Larger models are more **sample efficient** — for a fixed compute budget, training a larger model for fewer steps beats training a smaller model for more steps.
- **The core problem:** larger dense models perform better, **but** dense models activate **ALL** parameters for **every** token → scaling means massive compute, memory, and cost (e.g., GPT-3 175B ≈ 60k A100-days; a 1.6T dense model ≈ 540k A100-days).
- **Question:** how do we efficiently increase model size without proportionally increasing compute cost per token? → **Sparsely Activated Parameters**, i.e., MoE. Example: Switch Transformer scaled to 1.6 Trillion parameters while keeping *per-token* compute much lower than an equivalently-sized dense model.

5.2 What Is Mixture of Experts?

- MoE is a neural network architecture that **dynamically selects a subset** of specialized sub-models ("experts") to process each input.
- **Core mechanism:** replace each **Feed-Forward (FFN) module** in a Transformer block with **multiple** expert FFN layers — i.e., swap one dense FFN block for many FFN blocks ("experts"), of which only a few are active per token.

5.3 Components of an MoE Layer

Component	Role
Experts	individual sub-networks (usually FFNs), each somewhat specializing in different aspects of the data/context
Gating Network	determines the relevance of each expert for a given input; outputs a probability distribution over experts
Router	uses the gating network's output to direct the input to the chosen expert(s); ensures only a **subset** of experts is activated per token (this is what gives sparsity/efficiency)

- Each expert is **not** a full separate LLM — it's a sub-model that is part of the overall LLM's architecture (typically just the FFN sub-layer is "expert-ified"; attention layers are typically still shared/dense).
- **Alternate framing:** Dense FFN layer (all parameters always active) vs. Sparse layer (only a subset of "chopped up" expert pieces active per token) — each expert learns different specialized information during training; only the most relevant experts are used at inference for a given token/context.

5.4 The Math

Router / Gating:

1. Multiply input x by the router's learnable weight matrix W_g: $h = xW_g$
2. Apply **Softmax** to get a probability distribution $G(x)$ over experts.

Output (dense/soft MoE, all experts weighted):

$$y = G(x)_1 y_1 + G(x)_2 y_2 + \cdots + G(x)_n y_n = \sum_i G(x)_i \, E_i(x)$$

where $E_i(x)$ is expert i's output and $G(x)_i$ is the gate weight for expert i (its selection probability). This is a **weighted sum** — the gate values decide **how much to trust** each selected expert's output.

Sparse MoE — Greedy (Top-1) expert selection:

$$i^* = \arg\max_i \, G(x)_i \qquad y = G(x)_{i^*} \, E_{i^*}(x)$$

- Only the top-1 (or top-k) expert(s) actually process the token — gradients for all non-selected experts are **zero** for that token: $\dfrac{\partial L}{\partial E_i(x)} = 0$ for $i \neq i^*$.
- Most modern MoE LLMs use **top-k routing** (e.g., top-2), not strictly top-1.

MoE Layer Backpropagation (general, for the weighted-sum case):

$$\frac{\partial L}{\partial y_i} = \frac{\partial L}{\partial y} \cdot G(x)_i \qquad \frac{\partial L}{\partial G(x)_i} = \big(E_i(x)\big)^T \frac{\partial L}{\partial y}$$

- The gradient w.r.t. an expert's output is scaled by its gate weight; the gradient w.r.t. the gate weight depends on the expert's output — both the **experts** and the **gating network** are trained jointly, end-to-end.

5.5 Selection of Experts — Load Balancing Problem

- **Problem:** a naive router (just argmax of $G(x)$) tends to **collapse onto a few "favorite" experts** — because whichever experts happen to learn a bit faster early on get selected more, get more gradient updates, and become even better/more selected — a rich-get-richer feedback loop. Result: uneven expert usage, and some experts are barely trained at all — hurts both training and inference.
- **Intuition given in lecture:** like assigning students to group projects purely by skill score — without randomness, the top students always get picked, others never get a chance to improve.

Fix 1 — Noisy Top-K ("KeepTopK")

- Add **trainable Gaussian noise** to the router logits before selecting the top-K experts — this randomness gives lower-scoring experts an occasional chance to be selected and trained.
- **Mechanics of Top-K masking:** e.g., 5 experts, keep top K=2 scores; **all other scores are set to** $-\infty$ before softmax → those experts get probability 0 → they simply don't process that token.

Fix 2 — Auxiliary (Load-Balancing) Loss

- Added on top of the regular task loss to encourage a more **even distribution** of expert usage during training.
- **Step 1:** sum the router's gate values for each expert across the entire batch → gives an **importance score** per expert (how likely that expert is chosen, aggregated over the batch, independent of any single input).
- **Step 2:** compute the **Coefficient of Variation (CV)** of these importance scores — how different the scores are across experts.

  - **Low CV** = experts have similar importance → balanced (goal).
  - **High CV** = big imbalance → e.g., strongly biased toward one expert.

- **Step 3:** add a term to the loss that **penalizes high CV**, pushing training toward equal importance across experts.
- **Switch Transformer's simplified version:** instead of computing CV directly, weigh the **fraction of tokens dispatched** to each expert against the **fraction of router probability mass** assigned to that expert:

$$L_{aux} = \alpha \cdot N \cdot \sum_{i=1}^{N} f_i \cdot P_i$$

(conceptually: f_i = fraction of tokens routed to expert i, P_i = fraction of router probability mass on expert i; minimized when both are uniform across experts.)

  - α is a hyperparameter controlling the strength of this auxiliary loss: **too high** → overtakes/dominates the primary task loss; **too low** → does little for load balancing.

5.6 Expert Capacity & Token Dropping

- Imbalance isn't just about *which* experts get chosen — it's also about **how many tokens** each chosen expert receives. Solution: limit the number of tokens any single expert can process in a batch = **Expert Capacity**.
- **Token overflow:** if an expert (and its backup) is already at capacity, an incoming token assigned to it **cannot be processed by any expert** — it's passed straight to the next layer unchanged. This is called **token overflow / a dropped token**.
- **Capacity Factor:** modulates how much slack capacity each expert has relative to a perfectly uniform distribution — e.g., with 6 tokens in a batch and imperfect routing, some capacity slots go wasted (unfilled) while others overflow (tokens dropped) — capacity factor trades off wasted compute vs. dropped tokens.

"No Token Left Behind" — Two-Stage Routing

- **Stage 1:** route each token to its highest-probability expert.
- **Stage 2:** any token that got **dropped** (its first-choice expert was full) is routed to its **second-best** expert instead.
- This can be **iterated** (try 3rd choice, 4th choice, …) until (ideally) no token is left unprocessed.

5.7 Switch Transformer

- A **T5-style** (encoder-decoder) model that replaces the standard FFN layer with a **Switching Layer**.
- Uses **greedy routing to only 1 expert** (top-1 / hard routing) — simpler than top-k, and was shown to scale well (to 1.6T parameters).
- Trained on **TPUs**; uses a **statically compiled computational graph**, which is why expert capacity must be set as a fixed number ahead of time (facilitates efficient model-parallel placement — experts on different cores).

5.8 Expert Parallelism (distributed MoE training/inference)

- **All-to-All (A2A) communication:** tokens are sent from every GPU to whichever GPU(s) host their selected expert(s), so each expert receives exactly the tokens it needs to process.

- After the expert computes its output, a **second All-to-All** sends results back to the tokens' original GPUs so the rest of the Transformer layer (attention, etc., which is usually still dense/shared) can continue processing normally.
- This means MoE training/inference at scale requires careful **distributed systems** design — ties directly into Part 6 (Parallelism): MoE decks explicitly connect to **Pipeline Parallelism** (different layers on different devices) and **Tensor Parallelism** (column-wise / row-wise weight splitting) as complementary techniques for scaling *dense* portions of an MoE model, on top of expert parallelism for the sparse (expert) portions.

Part 5 Key Takeaways

- MoE swaps a dense FFN for many FFN "experts"; a **router/gating network** (softmax over a linear projection of x) picks which expert(s) process each token.
- Only a **sparse subset** of experts is active per token → much more capacity for the same *per-token* compute vs. an equally-large dense model.
- Naive routing **collapses** onto a few experts (rich-get-richer); fixed via **noisy top-k routing** and an **auxiliary (load-balancing) loss** based on the Coefficient of Variation of per-expert importance.
- **Expert Capacity** limits per-expert token load; overflow tokens get **dropped** (skip that layer) unless mitigated with **two-stage ("no token left behind") routing**.
- **Switch Transformer** = top-1 routing MoE, scaled to 1.6T params, TPU-trained with a static compute graph.
- Distributed MoE relies on **All-to-All** communication to ship tokens to the GPUs hosting their assigned experts, and back.

Part 6 — Parallelism & Distributed Training

6.1 Why Distributed Training Is Needed

Rough parameter-count formula for a decoder-only Transformer (with l layers, hidden dim h, vocab size v, sequence length s):

- Embeddings: $\approx vh$
- Attention + FFN blocks: $\approx 12lh^2$ (dominant term for large models)
- Output/probability projection: $\approx vh$
- **Total params** $\approx 12lh^2 + 2vh$

Memory cost example (GPT-3, 175B params):

- Model weights: $4 \times 175 \times 10^9$ bytes $\approx$ **700 GB** (using 4 bytes/param, FP32)
- Gradients: another **~700 GB**
- Optimizer states (e.g., Adam's momentum + variance): **~1.4 TB**
- **Total ≈ 2.8 TB** just for training state — far beyond a single GPU's memory (even an 80GB A100) → **must distribute** across many devices.

Chinchilla Law

The "RGB" of scaling a better LLM: **larger dataset + bigger model + longer training = better LLM** (compute-optimal scaling — Hoffmann et al.). LLaMA follows Chinchilla scaling. As models and datasets keep growing, **distributed training becomes increasingly critical**. (Example: Llama 3 trained using 24,000 GPUs.)

Practical Challenges of Massive-Scale Distributed Training

Two major categories: **stability** and **scalability**.

- **Stability:** GPU/hardware failures are common at scale — e.g., Meta's OPT-175B had 175+ job restarts from hardware failures over 2 months, wasting ~41,700 GPU-hours (>80% due to infrastructure failures).
- **Scalability:** communication overhead + GPU idle time hurt real-world throughput — GPUs often only achieve **30–55% of their peak FLOPs/s** during actual LLM training (vs. theoretical peak). If a GPU achieves 30% of its peak throughput, we say the system has "30% scalability." **Communication overhead is the top factor hampering scalability.**

6.2 The Three (+1) Core Parallelism Techniques — "3D Parallelism"

1. **Data Parallelism (DP)**
2. **Pipeline Parallelism (PP)**
3. **Tensor Parallelism (TP)** (Combined = **3D parallelism** — the most common way to organize large GPU clusters.)

6.3 Data Parallelism

- **Idea:** replicate the **full model** on every GPU; split the **batch** across GPUs (each GPU processes batch-size b/N for N GPUs).
- Implemented in modern data centers via **Ring-Allreduce** (federated learning historically used a parameter-server approach instead).
- **Pros:**
  - Cuts computation time by $\sim N$ fold (parallel batches).
  - Reduces activation memory by $\sim N$ fold (smaller per-GPU batch).
- **Cons:**

- **Does NOT save memory significantly** for the model itself — every GPU still needs a **full copy** of the model's parameters, gradients, and optimizer states. This is DP's core limitation for very large models.

6.4 Pipeline Parallelism

- **Idea: partition layers** across different GPUs (GPU1 has layers 1–k, GPU2 has layers k+1–2k, etc.).
- If partitioned **uniformly**, parameter/memory cost per GPU is cut by ~N fold.

The Idle-Time Problem (Naive / No Pipelining)

- Without careful scheduling, GPUs sit **idle** most of the time — e.g., GPU4 must wait for GPU1→2→3 to finish their forward pass before it can even start, and similarly waits during backward. **GPUs are not actually running in parallel** in the naive scheme.

The Fix — Micro-batching (the actual "pipeline")

- Split one batch into several **micro-batches**, and stream them through the pipeline (à la GPipe) so that while GPU1 works on micro-batch 2, GPU2 can simultaneously work on micro-batch 1, etc. — this **reduces (but does not fully eliminate)** idle time ("bubble").
- **Idle-time fraction formula (uniform partitions):** roughly $\frac{N-1}{m}$ where N = number of GPUs/pipeline stages, m = number of micro-batches. **More micro-batches → less idle time** (matches intuition).
- **Non-uniform partitions** (layers not evenly split across GPUs by compute cost) → **larger** idle-time fraction — harder to balance since some GPUs finish faster and wait longer.
- **Communication overhead in PP:** need to communicate **activations** (forward) and **activation gradients** (backward) — but only at **partition boundaries** (not every layer), and only for the **micro-batch's** activations (not the full batch) — relatively lightweight compared to Tensor Parallelism's communication.

Pipeline Parallelism — Summary

Pros	Cons
Cuts memory by ~N fold (uniform partition)	Introduces idle time (bubble) — cannot be fully removed
Accelerates training via micro-batch parallelism	Difficult to partition layers uniformly (different layer costs)
Lower comm. overhead than tensor parallelism (only at partition boundaries, only activations, only micro-batch-sized)	

6.5 Tensor Parallelism

- **Idea:** partition **individual tensors/weight matrices** themselves across GPUs (not whole layers) — save memory at a finer grain than pipeline parallelism.
- **Two partitioning styles:**
  - **Column-wise splitting:** split a weight matrix by columns across GPUs.
  - **Row-wise splitting:** split a weight matrix by rows across GPUs.
- **Communication primitives used:**
  - **All-Gather** — used to reconstruct full outputs after a column-parallel split.
  - **All-Reduce** — used to sum partial outputs after a row-parallel split.

  - **Reduce-Scatter** — used in some configurations combining gather+reduce.
- Concretely (2-GPU MLP example from the deck): with **column-wise then row-wise** splits chained across two linear layers, you can arrange computation so that **no idle time** occurs — only communication and computation time exist, but note that **almost every layer incurs a communication step** (unlike pipeline parallelism, which only communicates at partition boundaries).

Tensor Parallelism — Summary

Pros	Cons
Cuts memory by ~N fold (uniform partition)	Introduces **more** communication than pipeline parallelism
Accelerates training via parallelism	Partitioning affects comm. cost — non-trivial to find a good partition scheme
No idle time (unlike pipeline parallelism)	

6.6 3D Parallelism — Putting It All Together

Technique	Saves	Comm. cost	Idle time?	Sensitivity
Data Parallelism	Reduces *time*, but does **not** meaningfully reduce memory (keeps a full model copy per GPU)	Moderate (gradient all-reduce)	No	—
Pipeline Parallelism	Reduces **memory** by N-fold (uniform split)	**Less** comm. (only at partition boundaries, activations only)	**Yes** — introduces idle "bubble"	Sensitive to how **layers** are partitioned
Tensor Parallelism	Reduces **memory** by N-fold (uniform split)	**Big** comm. cost (nearly every layer)	**No** idle time	Sensitive to how **tensors** are partitioned

- **3D parallelism** = combining Data + Pipeline + Tensor parallelism to jointly manage compute, memory, and communication trade-offs across a large GPU cluster. Different combinations produce very different overall training performance — but since each individual mechanism is well-understood, the total training time **can, in principle, be fully modeled/computed** in advance.

Part 6 Key Takeaways

- GPT-3-scale training needs ~2.8TB just for weights+gradients+optimizer state — far beyond one GPU → distribution is mandatory.
- **Data Parallelism:** replicate model, split batch — cuts time, **not** memory (full model copy needed per GPU).
- **Pipeline Parallelism:** split layers across GPUs — cuts memory, introduces **idle-time bubbles**, mitigated (not eliminated) by micro-batching; **lighter** communication (only at boundaries).
- **Tensor Parallelism:** split individual weight matrices (column-/row-wise) across GPUs — cuts memory, **no idle time**, but **heavier** communication (nearly every layer: All-Gather / All-Reduce).

- **3D Parallelism** = DP + PP + TP combined — the standard approach for training today's largest models.
- Real-world GPUs often hit only 30–55% of peak FLOPs due to communication overhead and idle time — communication is the #1 scalability bottleneck.

Part 7 — Rapid-Fire Formula & Definition Cheat Sheet

Concept	Formula / Definition
RMSNorm	$x/\sqrt{\frac{1}{n}\sum x_i^2 + \epsilon} \cdot \gamma$ (no mean subtraction, unlike LayerNorm)
SwiGLU	$\text{Swish}(xW_1) \otimes (xW_2)$; $\text{Swish}(x) = x\sigma(x)$
Sinusoidal PE	$PE_{(pos,2i)} = \sin(pos/10000^{2i/d})$, $PE_{(pos,2i+1)} = \cos(\cdot)$ — **absolute** position
RoPE	rotates Q,K by position-dependent angle → dot product depends only on **relative** distance $m - n$
Bradley-Terry	$p^*(y_1 \succ y_2 \mid x) = \sigma(r^*(x, y_1) - r^*(x, y_2))$
Reward Model loss	$-\mathbb{E}[\log \sigma(r_\varphi(x, y_w) - r_\varphi(x, y_l))]$
RLHF/PPO objective	$\max_\theta \mathbb{E}[r(x, y)] - \beta \cdot D_{KL}(\pi_\theta \| \pi_{ref})$
Advantage	Reward − Baseline (PPO: learned value fn.) / Reward − group mean (GRPO)
PPO-Clip	$\mathbb{E}[\min(r_t(\theta)A_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t)]$
DPO	supervised rewrite of RLHF loss; no reward model, no RL loop, needs only π_{ref}
GRPO	PPO but baseline = mean reward of a **sampled group** (no value model)
RLVR	reward from ground-truth answer or rule-based checker (math/code) — no learned RM
MoE gating	$G(x) = \text{softmax}(xW_g)$; output $y = \sum_i G(x)_i E_i(x)$ (or top-k sparse)
MoE aux loss (Switch)	$L_{aux} = \alpha N \sum_i f_i P_i$ (f_i=frac. tokens routed, P_i=frac. router prob mass)
Pipeline idle-time fraction	$\approx (N-1)/m$ (N=GPUs, m=micro-batches)
GPT-3 memory	weights 700GB + grads 700GB + optimizer 1.4TB ≈ 2.8TB total

Part 8 — MCQ Bank (with Answers)

Q1. Which normalization technique removes the mean-subtraction (re-centering) step used in LayerNorm? A) BatchNorm B) RMSNorm C) GroupNorm D) InstanceNorm **Answer: B.** RMSNorm keeps only re-scaling (divides by RMS), dropping re-centering — cheaper and used in most modern LLMs.

Q2. BatchNorm is less suited to sequence models mainly because: A) It cannot be backpropagated through B) It depends on batch statistics, which are unstable with variable-length sequences/small batches C) It doesn't use learnable parameters D) It only works with images **Answer: B.**

Q3. SwiGLU improves on plain GLU by: A) Replacing the sigmoid gate with Swish, avoiding gate saturation B) Removing the gate entirely C) Adding a second gate D) Using ReLU instead of a gate **Answer: A.**

Q4. What is the key limitation of sinusoidal (absolute) positional encoding that RoPE fixes? A) It's too computationally expensive B) It cannot handle sequences longer than 512 tokens C) It encodes absolute position rather than relative distance between tokens D) It only works for encoder models **Answer: C.**

Q5. In RoPE, the dot product between rotated Q and K vectors at positions m and n depends on: A) $m + n$ B) $m \times n$ C) $m - n$ (their relative distance) D) Only m **Answer: C.**

Q6. The KV cache in autoregressive decoding primarily saves compute by: A) Reducing the model's parameter count B) Avoiding recomputation of Keys/Values for previously generated tokens C) Compressing the vocabulary D) Removing the need for positional encoding **Answer: B.**

Q7. Grouped-Query Attention (GQA) is best described as: A) Every head has its own K/V (like standard MHA) B) All heads share one single K/V (like MQA) C) Heads are divided into groups, each group sharing its own K/V D) No K/V caching at all **Answer: C.**

Q8. PagedAttention (used in vLLM) manages the KV cache by: A) Storing everything in one large contiguous block per sequence B) Splitting KV cache into fixed-size blocks with a page table, similar to OS virtual memory C) Deleting old tokens' KV entries after each step D) Avoiding KV cache entirely **Answer: B.**

Q9. FlashAttention speeds up attention primarily by: A) Reducing the number of floating point operations (FLOPs) B) Approximating the softmax C) Reducing memory accesses to slow HBM via tiling and recomputation (exact computation) D) Using a smaller attention window **Answer: C.** (Important: FlashAttention gives the *exact* same output — it's not an approximation.)

Q10. Speculative decoding uses: A) Only the large target model, sampled twice B) A small draft model to propose tokens, verified by a larger target model in one pass C) Two large models trained jointly from scratch D) A rule-based decoder instead of a neural model **Answer: B.**

Q11. Multi-Head Latent Attention (MLA) reduces KV-cache memory by: A) Sharing K/V heads across groups like GQA B) Compressing K/V into a lower-dimensional latent vector before caching, projecting back up when needed C) Deleting the K cache and keeping only V D) Using sliding windows **Answer: B.**

Q12. What is the main purpose of pretraining? A) Teach the model to refuse harmful requests B) Build broad language understanding/generation via self-supervised next-token prediction on massive raw text C) Directly optimize for human preference D) Compress the model **Answer: B.**

Q13. Why does SFT need comparatively little data (e.g., ~13K examples for InstructGPT) despite pretraining needing billions of tokens? A) SFT uses a different, more efficient optimizer B) The model already has the knowledge from pretraining; SFT only teaches a new interaction *mode*, not new knowledge C) SFT data is compressed D) SFT doesn't actually change the weights **Answer: B.**

Q14. Over-training on SFT data risks: A) Underfitting B) The model memorizing exact response patterns (e.g., verbatim refusal phrases) instead of generalizing C) Catastrophic forgetting of the tokenizer D) Vanishing gradients **Answer: B.**

Q15. Which benchmark specifically measures multi-step grade-school math word-problem solving? A) MMLU B) ARC-Challenge C) GSM8K D) HumanEval **Answer: C.**

Q16. Which benchmark measures the ability to write correct Python functions from natural-language specs? A) MMLU B) HumanEval C) GSM8K D) ARC-Challenge **Answer: B.**

Q17. A key flaw of voting/leaderboard-based LLM evaluation is: A) It's too slow to compute B) Safe refusals often get rated lower than helpful-but-unsafe answers ("safety penalization") C) It cannot be automated D) It only works for code models **Answer: B.**

Q18. Why is preference tuning (comparisons) generally preferred over collecting more demonstration data? A) Comparisons require less human effort/are more reliable than writing an ideal answer from scratch — recognition is cheaper than generation B) Comparisons are always free C) Demonstrations can't be labeled at all D) Comparisons don't require any human involvement **Answer: A.**

Q19. In the Bradley-Terry model, if $\Delta = r^*(x, y_1) - r^*(x, y_2) = 0$, then $p^*(y_1 \succ y_2|x) =$ A) 0 B) 1 C) 0.5 D) Undefined **Answer: C.**

Q20. The reward model is trained using a loss that is: A) Mean squared error on absolute scores B) The same logistic (negative log-likelihood) loss as Bradley-Terry MLE, over preference pairs C) Cross-entropy over the vocabulary D) A pure L1 loss **Answer: B.**

Q21. What does the "HF" in RLHF specifically refer to in the RM's training data? A) High-Frequency tokens B) Human rating used as the label for preference pairs C) Hidden Features D) Hyperparameter Fitting **Answer: B.**

Q22. In the RLHF/PPO objective, the KL-divergence penalty term exists to: A) Increase reward directly B) Prevent the policy from drifting too far from the reference model (avoiding reward hacking / diversity collapse) C) Speed up convergence with no other purpose D) Replace the reward model **Answer: B.**

Q23. PPO actually optimizes based on: A) Raw reward only B) Advantage (reward minus a learned baseline/value function) C) Only the KL term D) Pure loss from the SFT stage **Answer: B.**

Q24. In PPO-Clip, when the policy ratio $r_t(\theta)$ becomes very large for a positive-advantage action, the clipping mechanism: A) Increases the reward further B) Prevents over-boosting that action's probability by choosing the clipped (smaller) term via min() C) Removes the value function D) Has no effect **Answer: B.**

Q25. In modern PPO implementations, the KL-penalty term is typically computed with respect to: A) The old policy from the previous iteration B) The reference/base model C) A randomly initialized model D) The reward model **Answer: B.**

Q26. What is the main limitation of Best-of-N (BoN) that motivates DPO? A) BoN requires too much training data B) BoN is costly at inference time (must generate + score N candidates per query) C) BoN cannot use a reward model D) BoN only works for code generation **Answer: B.**

Q27. DPO's key innovation is that it: A) Trains a much larger reward model than PPO B) Requires a separate value function C) Rewrites the RLHF objective into a supervised loss directly on preference data, with no separate reward model or RL loop D) Uses greedy decoding to select training examples **Answer: C.**

Q28. Compared to PPO/RLHF, DPO needs which extra model(s) besides the policy being trained? A) None B) Only the reference/base model C) Reward model + value model D) Four extra models **Answer: B.**

Q29. GRPO computes its "advantage" as: A) Reward minus a separately-trained value/critic model's estimate B) Reward minus the average reward of a group of sampled responses to the same prompt C) Raw reward with no baseline D) The KL divergence itself **Answer: B.**

Q30. Why is GRPO considered more memory-efficient than PPO? A) It uses fewer sampled responses B) It eliminates the need for a separate value/critic model C) It removes the reward model entirely D) It doesn't require a reference model **Answer: B.**

Q31. RLVR (Reinforcement Learning with Verifiable Rewards) is most applicable to: A) Open-ended creative writing B) Domains with a ground-truth answer or rule-based correctness check, e.g., math and coding C) Sentiment analysis only D) Any task, equally **Answer: B.**

Q32. In MoE, the "router"/"gating network" is responsible for: A) Computing the final loss B) Outputting a probability distribution over experts and directing tokens to the selected expert(s) C) Normalizing the input embeddings D) Applying positional encoding **Answer: B.**

Q33. MoE replaces which component of a standard Transformer block with multiple expert copies? A) The self-attention layer B) The Feed-Forward Network (FFN) layer C) The embedding layer D) The output softmax **Answer: B.**

Q34. Why does naive (argmax-only) expert routing cause a load-balancing problem? A) Because experts are too large B) Because faster-learning experts get selected more often, creating a rich-get-richer feedback loop, leaving other experts under-trained C) Because gradients cannot flow into the router D) Because there are too few tokens **Answer: B.**

Q35. One fix for MoE load-balancing is: A) Removing the router entirely B) Adding trainable Gaussian noise before top-k selection ("noisy top-k") plus an auxiliary load-balancing loss C) Using only one expert always D) Increasing the learning rate for the reward model **Answer: B.**

Q36. In the auxiliary load-balancing loss, a LOW coefficient of variation (CV) across expert importance scores means: A) Severe imbalance, biased toward one expert B) Experts have similar importance — a balanced, desired outcome C) The router has collapsed D) Training has diverged **Answer: B.**

Q37. "Token overflow" in MoE occurs when: A) A token's embedding is too large B) A token's selected expert(s) are already at full Expert Capacity, so the token isn't processed by any expert and is passed to the next layer C) The batch size is too small D) The router assigns negative probabilities **Answer: B.**

Q38. The "No Token Left Behind" strategy addresses token overflow by: A) Increasing the model size B) Two-stage routing — sending dropped tokens to their second-best expert (iterated if needed) C) Deleting overflowing tokens permanently D) Disabling the router **Answer: B.**

Q39. The Switch Transformer uses: A) Top-2 routing with a dense value function B) Greedy top-1 routing, trained on TPUs with a statically compiled graph C) No routing at all — all experts always active D) Data parallelism only, no expert parallelism **Answer: B.**

Q40. In distributed MoE, All-to-All (A2A) communication is used to: A) Synchronize optimizer states only B) Send tokens from every GPU to the GPU(s) hosting their selected expert(s), and send results back C) Broadcast the loss value to all GPUs D) Replace the router **Answer: B.**

Q41. Data Parallelism's main limitation for very large models is: A) It doesn't reduce training time at all B) It requires a full copy of the model (params, gradients, optimizer states) on every GPU — doesn't meaningfully reduce memory C) It cannot be combined with other parallelism types D) It only works with a single GPU **Answer: B.**

Q42. Pipeline Parallelism partitions a model by: A) Splitting individual weight matrices across GPUs B) Assigning different layers to different GPUs C) Replicating the full model on every GPU D) Splitting the vocabulary **Answer: B.**

Q43. The main downside of naive (non-micro-batched) pipeline parallelism is: A) Excessive communication of full-batch model weights B) Significant GPU idle time, since GPUs must wait on

each other sequentially C) It cannot reduce memory usage D) It requires more GPUs than tensor parallelism **Answer: B.**

Q44. According to the idle-time formula for pipeline parallelism (≈ (N−1)/m), increasing the number of micro-batches m: A) Increases idle time B) Decreases idle time C) Has no effect on idle time D) Only affects tensor parallelism **Answer: B.**

Q45. Tensor Parallelism, compared to Pipeline Parallelism, generally has: A) Less communication overhead but more idle time B) More communication overhead (nearly every layer) but no idle time C) Identical communication and idle-time profiles D) No use of All-Reduce or All-Gather **Answer: B.**

Q46. "3D Parallelism" refers to combining: A) Data, Pipeline, and Tensor parallelism B) Only Data and Tensor parallelism C) Three copies of the same GPU cluster D) Expert, Sequence, and Context parallelism only **Answer: A.**

Q47. Per the Chinchilla Law, a better LLM comes from: A) Bigger model only B) Larger dataset + bigger model + longer training, combined (compute-optimal scaling) C) Longer training only, regardless of data/model size D) Smaller model with more GPUs **Answer: B.**

Q48. A GPU achieving only 30% of its peak FLOPs/s during LLM training indicates: A) 30% scalability — most throughput lost to communication overhead / idle time B) The GPU is broken C) The model has converged D) Data parallelism is not being used **Answer: A.**

Part 9 — Short Theory / Short-Answer Questions (with Model Answers)

Q1. Explain why RMSNorm removes mean-centering, and why this doesn't hurt performance in LLMs. *Model answer:* In Q/K projections used for attention, dot products naturally produce both positive and negative values; subtracting the mean (as LayerNorm does) can strip out useful absolute information, couples feature representations unnecessarily, and adds computation. RMSNorm keeps only the re-scaling (variance-normalization) property of LayerNorm — shown empirically to preserve training stability/generalization while being cheaper, which matters a lot at the FLOP/latency scale of billion-parameter models.

Q2. Why is self-attention "permutation invariant," and what problem does this create? *Model answer:* Self-attention computes attention scores from content (Q·K similarity) alone, with no inherent sense of token order — shuffling the input tokens produces the same set of pairwise relationships, just reassigned. This means the model would treat a sentence as a "bag of words" without some explicit mechanism (positional encoding) to inject order information.

Q3. Contrast absolute positional encoding with RoPE. *Model answer:* Absolute (sinusoidal) PE adds a fixed position-dependent vector to each token embedding — the model effectively "memorizes" that a token is at position 23, but has no explicit signal about relative distances between tokens, and position information is disentangled from the attention computation itself. RoPE instead rotates the Query and Key vectors by an angle proportional to their position; because rotation preserves length but changes orientation, the resulting Q·K dot product depends only on the *difference* in rotation angles — i.e., the *relative* distance between tokens — which is more aligned with how language dependencies actually work (relational, not tied to absolute index).

Q4. Explain the KV cache and why it's needed. *Model answer:* During autoregressive generation, at each step the model only needs to compute Q, K, V for the newly generated token; the Keys and Values of all previously generated tokens are unchanged and can be cached. Without caching, the model would recompute K/V for the entire prefix at every single generation step — wasteful, since the prefix's K/V are static after being computed once. The KV cache trades memory (storing past K/V) for a large reduction in redundant computation.

Q5. Explain the trade-off represented by GQA vs. MHA vs. MQA. *Model answer:* MHA gives each attention head its own K/V — highest quality but largest KV cache/memory-bandwidth cost. MQA shares a single K/V across all heads — smallest KV cache, but can hurt model quality from too much sharing. GQA groups heads and shares K/V within each group — a tunable middle ground that recovers most of MHA's quality while significantly shrinking the KV cache versus full MHA, which is why it became the standard in modern LLMs.

Q6. What problem does FlashAttention solve, and how (without changing the output)? *Model answer:* Standard attention implementations are bottlenecked by repeated reads/writes between fast SRAM and slow HBM memory when computing and storing the full attention matrix, not by the FLOPs themselves. FlashAttention uses **tiling** (breaking Q/K/V into SRAM-sized blocks so each value is read from HBM only once) and **recomputation** (recomputing intermediate values during the backward pass instead of storing them) to minimize HBM accesses, while still computing the mathematically *exact* same attention output — it is not an approximation.

Q7. Why does SFT need far less data than pretraining, and what exactly does SFT "teach" the model? *Model answer:* Pretraining already gives the model broad world/language knowledge from massive raw text. SFT doesn't need to teach new knowledge — it teaches a new *mode of interaction*: read an instruction, produce a direct, appropriately-formatted response, instead of continuing text like a web-page completion. This is a comparatively simple behavioral shift, so a small (10K–100K example), high-quality, diverse dataset is sufficient — as opposed to the trillions of tokens needed to build the underlying knowledge itself.

Q8. What is the "asymmetry" argument for why RLHF (preference-based training) is used instead of only collecting more demonstrations? *Model answer:* Recognizing which

of two responses is better is cognitively easier, faster, and more consistent for a human than generating the single best possible response from scratch. This asymmetry — recognition is cheaper than generation — means preference (comparison) data can be collected more reliably and at greater scale than perfect demonstrations, which is the economic/statistical foundation for RLHF.

Q9. State the Bradley-Terry preference model and explain what happens as $\Delta \to +\infty$ or $\Delta \to -\infty$. *Model answer:* $p^*(y_1 \succ y_2|x) = \sigma(r^*(x, y_1) - r^*(x, y_2))$. As Δ (the reward gap) $\to +\infty$, $\sigma(\Delta) \to 1$ (near-certain preference for y_1); as $\Delta \to -\infty$, $\sigma(\Delta) \to 0$ (near-certain preference for y_2); at Δ=0, $\sigma(0) = 0.5$ (a toss-up). This lets the model represent graded confidence/noise in human preference judgments rather than a hard binary rule.

Q10. Why do we constrain the RL-trained policy to stay close to the reference model (via KL penalty) rather than purely maximizing reward? *Model answer:* The reward model is an imperfect proxy trained on a limited set of natural outputs. Pure reward maximization can lead to (a) reward hacking — outputs that fool the RM into high scores without being genuinely good (e.g., confident hallucinations), and (b) diversity collapse — since many outputs can be "correct" for a prompt (e.g., many valid poems), pure maximization can collapse the policy onto one narrow output style. Penalizing KL-divergence from the reference model keeps the policy close to a known-reasonable distribution, mitigating both failure modes.

Q11. What does "advantage" mean in PPO, and why use it instead of raw reward? *Model answer:* Advantage ≈ Reward − Baseline, where the Baseline is a learned value function estimating the *expected* reward for the current state under the current policy. Using advantage tells the model whether an action was better or worse than what was *already expected*, which reduces the variance of the policy-gradient estimate compared to using raw reward directly, leading to more stable training.

Q12. Explain the purpose of PPO-Clip's clipping mechanism. *Model answer:* PPO-Clip clips the ratio between the new and old policy's probability for an action, bounding it to $[1 - \epsilon, 1 + \epsilon]$, and takes the minimum of the clipped and unclipped surrogate objective. This prevents any single update from moving the policy too far in one step — protecting against a "big jump" that could destabilize training or collapse performance — while still allowing normal-sized improvements when the ratio stays within the trust region.

Q13. Compare PPO and DPO in terms of models required and training paradigm. *Model answer:* PPO-based RLHF is multi-stage and needs up to four models: the policy being trained, a value/baseline model, a reward model, and a reference/base model — an RL training loop with sampling. DPO reformulates the same underlying objective into a single supervised loss computed directly on (prompt, chosen, rejected) preference triples, needing only the policy and the reference model — no reward model, no RL sampling loop — making it simpler to implement, though performance relative to PPO is task-dependent with no universal winner.

Q14. How does GRPO's advantage estimate differ from PPO's, and why was this useful for training reasoning models? *Model answer:* PPO estimates advantage using a separately trained value/critic model's prediction as the baseline. GRPO instead samples a *group* of responses to the same prompt and uses the group's **mean reward** as the baseline — advantage = individual reward − group mean. This removes the need to train and maintain a value model (which is especially hard to get right for long reasoning chains), and directly exploits the fact that for reasoning tasks, you can cheaply generate many candidate solutions to the same problem and compare them against each other — making training simpler and more memory-efficient.

Q15. What does RLVR add on top of GRPO/PPO-style training, and where is it most useful? *Model answer:* RLVR replaces the learned (and imperfect) reward model with a reward computed directly from a **ground-truth answer** or a **rule-based verifier** (e.g., checking if a math answer matches, or if generated code passes unit tests). It's most useful in domains with objectively verifiable correctness — math and coding — where it avoids the noise/hackability of a learned reward model.

Q16. Explain the MoE load-balancing problem and describe two mitigations. *Model answer:* A naive router tends to repeatedly select whichever experts happen to be slightly ahead early in training (they get more gradient signal, become even better, get selected even more) — a rich-get-richer collapse that leaves other experts undertrained and creates both training and inference issues. Two mitigations: (1) **Noisy top-k routing** — add trainable Gaussian noise to the router's logits before selecting the top-k experts, giving lower-scoring experts an occasional chance to be picked and trained; (2) an **auxiliary load-balancing loss** that penalizes high variance (coefficient of variation) in per-expert importance scores aggregated over a batch, explicitly pushing the router toward more even expert usage.

Q17. What is "Expert Capacity" in MoE, and what happens on overflow? *Model answer:* Expert Capacity caps the number of tokens any single expert can process within a batch, preventing extreme overloading of popular experts. If a token's assigned expert(s) are already at capacity, that token is "dropped" — it is not processed by any expert for that layer and is simply passed through unchanged to the next layer ("token overflow"). The "No Token Left Behind" strategy mitigates this via two-stage (or iterated) routing: send overflowed tokens to their next-best expert instead of dropping them entirely.

Q18. Why does Data Parallelism fail to solve the memory problem for very large models? *Model answer:* In Data Parallelism, each GPU holds a **full replica** of the model's parameters, gradients, and optimizer states — it only splits the *batch*, not the *model*. So even with many GPUs, every single GPU must still be able to fit the entire model in memory, which becomes impossible for models with hundreds of billions of parameters — hence the need for Pipeline and/or Tensor parallelism, which actually split the model itself across devices.

Q19. Why does Tensor Parallelism have no idle time, while Pipeline Parallelism does? *Model answer:* In Tensor Parallelism, all GPUs work simultaneously on different slices of the *same* layer's computation for every input, communicating (All-Gather/All-Reduce) frequently but continuously — no GPU sits idle waiting on another to finish an earlier stage. In Pipeline Parallelism, different GPUs own different *layers*, so a GPU handling a later layer must wait for earlier GPUs to finish processing that micro-batch first — creating unavoidable "bubbles" of idle time, only partially mitigated (not eliminated) by using many small micro-batches.

Q20. Why is communication overhead described as "the top factor hampering scalability" in distributed LLM training? *Model answer:* Even though modern GPUs have very high theoretical peak FLOPs, real-world large-scale training often only achieves 30–55% of that peak, because GPUs spend significant time either communicating (synchronizing gradients, activations, or tensor-parallel partial results) or sitting idle waiting for other GPUs (as in pipeline parallelism) rather than doing useful computation. This gap between theoretical and achieved throughput is dominated by communication/coordination overhead rather than by compute limitations, making communication-efficient parallelism strategies critical.

Part 10 — Tricky Questions, Gotchas & Common Confusions

These are the distinctions that quizzes love to test — read carefully.

1. **"FlashAttention reduces FLOPs" — FALSE.** FlashAttention reduces **memory accesses** (HBM reads/writes), not the number of floating-point operations. The computation is mathematically **exact**, not approximate.
2. **RoPE "adds" a positional vector — FALSE.** Sinusoidal PE *adds* a vector to the embedding. RoPE instead **rotates** the Q and K vectors — it's multiplicative/rotational, not additive, and it's applied inside the attention mechanism (to Q/K), not just added to the input embedding once.
3. **GQA vs MQA — don't confuse them.** MQA = **one single** shared K/V for *all* heads. GQA = heads split into **groups**, each group has its **own** shared K/V (a middle ground between MHA's "every head has its own" and MQA's "everyone shares one").
4. **RMSNorm still has a learnable parameter** — it's not "no learnable parameters," it just removes the *mean-subtraction* step (re-centering), not the scale parameter γ.
5. **SFT and RLHF are NOT mathematically the same objective**, even though both use a next-token-prediction-style mechanism under the hood in different ways. SFT = imitation (maximize likelihood of exact demonstrations). RLHF = preference optimization (maximize *relative* reward vs. alternatives, subject to a KL constraint) — the slides explicitly stress "these sound similar but are mathematically different objectives."
6. **Preference tuning does NOT teach new knowledge/reasoning.** It reshapes/aligns *existing* behavior of an already pretrained+SFT model — a common trick question tests whether you think RLHF "teaches the model new facts" (it doesn't).
7. **The KL penalty in modern PPO is typically w.r.t. the reference/base model ("ref"), NOT the "old" policy from the previous iteration** — don't mix up `old` (used for the importance-sampling ratio r_t) vs. `ref` (used for the KL constraint).
8. **DPO removes the reward model AND the RL loop — but NOT the reference model.** A common trap is saying DPO needs "just the policy" — it still needs π_{ref} to compute its implicit reward / loss.
9. **GRPO removes the value/critic model, not the reward model.** GRPO still needs *some* reward signal per sampled response (from a reward model, or — in RLVR settings — from a rule-based/ground-truth checker) to compute each response's reward before comparing it to the group mean. Don't confuse "no value function" with "no reward at all."
10. **Best-of-N is NOT a training method** — it's an **inference-time** trick (generate N, score with RM, keep the best). It doesn't change the underlying model's weights at all. This is exactly why it's "costly at inference time" — the cost is paid every single query, forever, not once during training.
11. **MoE experts are NOT full separate LLMs.** Each "expert" is typically just a Feed-Forward sub-network — replacing the FFN portion of a Transformer block. The attention layers are usually still shared/dense across the whole model, not "expert-ified."
12. **Top-1 (greedy) MoE routing has ZERO gradient for non-selected experts on a given token** — $\partial L/\partial E_i(x) = 0$ for $i \neq i^*$. This is a common trick — students think all experts always get *some* gradient; they don't, under hard/greedy routing.
13. **Auxiliary loss ≠ the main task loss.** It's an additional term specifically for load balancing; too high an α and it can actually **hurt** model quality by overpowering the main task loss — it's not "free," it's a hyperparameter trade-off.
14. **Token "dropping" in MoE ≠ token deletion from the sequence.** A dropped/overflowed

token is simply **not processed by any expert at that layer** — it's passed through unchanged to the next layer; it's not removed from the sequence entirely.

15. **Data Parallelism does not reduce per-GPU memory for the model** — a very common misconception. It reduces *time* (via parallel batches) and *activation* memory, but every GPU still holds a full copy of parameters/gradients/optimizer states. Only Pipeline and Tensor parallelism actually shard the *model* itself.
16. **Pipeline Parallelism has LESS communication than Tensor Parallelism, but introduces idle time; Tensor Parallelism has NO idle time but MORE communication.** These are opposite trade-offs — a classic "match the technique to its property" quiz question.
17. **Switch Transformer uses top-1 (greedy), not top-2 or soft/weighted routing** across all experts — a specific detail worth memorizing since GRPO/DeepSeek-style top-k (k>1) is different and more common in newer models.
18. **Chinchilla scaling is about the balance of model size vs. data size vs. training length together** — not simply "bigger model is always better." A common trap answer is "just make the model bigger"; the actual principle is *compute-optimal* balance across all three ("RGB": dataset, model, training time).
19. **Evaluation leaderboards can penalize genuinely safe behavior** ("safety penalization" — a safe refusal can score lower than a harmful-but-helpful-sounding answer) — this is a subtle but frequently-tested point about the *limits* of preference-based evaluation, not just its benefits.
20. **RMSNorm vs LayerNorm exam trap:** LayerNorm normalizes per-example across the *feature* dimension (not the batch dimension — that's BatchNorm). Don't confuse "LayerNorm doesn't depend on batch" with "LayerNorm normalizes across the batch" — it's the opposite; LayerNorm is batch-independent precisely *because* it normalizes within each example's own features.

Final Pre-Quiz Checklist

- ☐ Can you state the Bradley-Terry formula and explain $\Delta=0$ / $\Delta>0$ / $\Delta<0$?
- ☐ Can you write the PPO objective (reward $-\ \beta\cdot$KL) and explain why the KL term exists?
- ☐ Can you list what's different between PPO, DPO, and GRPO (models needed, baseline used)?
- ☐ Can you explain RMSNorm vs LayerNorm vs BatchNorm in one sentence each?
- ☐ Can you explain why RoPE encodes *relative* not *absolute* position?
- ☐ Can you explain MQA → GQA → MLA as a progression of KV-cache memory savings?
- ☐ Can you explain FlashAttention's two optimizations (tiling, recomputation) and that it's exact?
- ☐ Can you explain MoE routing, load balancing (noisy top-k + auxiliary loss), and expert capacity/overflow?
- ☐ Can you contrast Data / Pipeline / Tensor parallelism on memory savings, idle time, and communication cost?
- ☐ Can you explain why SFT needs little data but RLHF/preference data is about *comparisons* being cheaper than *generation*?

Good luck on the quiz!